

Relatório Atividade 5

Otimizações em python

Integrantes: Luiz Fellipe Catuzzi 11871198

Juan Henriques Passos 15464826

Fernando Valentim Torres 15452340

Impacto das sugestões para otimização em python

A versão em Python de um código que você desenvolveu em outras atividades da disciplina.

O código utilizado foi o referente a atividade 3 da aula 4 que consistia em desenvolver um único programa contendo 3 algoritmos de ordenação definidos em funções e analisar o tempo gasto em cada algoritmo. Para isso, antes da chamada de cada função deve-se carregar o mesmo vetor a ser ordenado e deve-se executar uma função para limpar a cache para que isso não tenha influência no tempo de execução.

Processos executados:

- Tradução C para Python.
- Implementado dois códigos(sem e com otimização).
- Utilizado timeit para pegar a média do tempo das ordenações.

Usando o timeit, cada função foi executada 100 vezes com um array de 1000 valores a serem ordenados. Os tempos de execução obtidos com o código sem e com otimização são respectivamente:

```
Código referente ao teste sem otimização.  
Iniciando benchmark com 100 iterações.  
Tamanho do Array: 1000 elementos.  
Executando benchmarks...  
  
=== Resultados Finais (Média de tempo) ===  
Bubble Sort: 0.038472813999978824 segundos  
Selection Sort: 0.02479679899988696 segundos  
Insertion Sort: 0.00018668499993509613 segundos  
Tempo execução do programa: 8.109078645706177 segundos
```

```
Código referente ao teste com otimização.  
Iniciando benchmark com 100 iterações.  
Tamanho do Array: 1000 elementos.  
Executando benchmarks...  
  
=== Resultados Finais (Média de tempo) ===  
Bubble Sort: 0.024087 segundos  
Selection Sort: 0.017207 segundos  
Insertion Sort: 0.000229 segundos  
Built-in arr.sort(): 0.000004 segundos  
  
Tempo execução do programa: 6.20 segundos
```

A função de limpar a cache utilizava variáveis globais que continham o tamanho da cache e um buffer desse tamanho, no entanto, o acesso a variáveis globais *LOAD_GLOBAL* é, computacionalmente, menos eficiente que *LOAD_FAST* que é o carregamento de variáveis locais.

Antes:

```
global dummy_buffer, LARGEST_CACHE_SZ
```

Depois:

```
def clean_cache(buffer, size):
```

- Otimização de **atribuição múltipla para troca**:

Nos algoritmos como bubblesort e o selection sort, é realizado uma atribuição múltipla para fazer as trocas dos elementos ao invés de usar uma variável temporária. Tal otimização destaca-se no tempo de execução menor tanto no bubble quando do selection em comparação ao sem otimização.

Antes:

```
temp = arr[min_idx]
arr[min_idx] = arr[i]
arr[i] = temp
```

Depois:

```
arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

- Otimização para **limitar busca de métodos e lista de compressão**

Armazena-se o 'random.randint' em uma variável local (_randint) para evitar a busca de atributo 'random.randint' a cada iteração do loop (lista de compressão). Além disso, é usado a list compression porque é uma forma mais rápida, concisa e idiomática em python para criar listas, comparada a um loop 'for' com '.append()'

Antes:

```
arr = []
i = 0
while i < n:
    arr.append(random.randint(0, 9999))
    i = i + 1
return arr
```

Depois:

```
_randint = random.randint
return [_randint(0, 9999) for _ in range(n)]
```

- Otimização de **Operação com String**

Uso de str.join() melhor que concatenação com '+'

Antes:

```
output = "Array: "  
i = 0  
while i < len(arr):  
    output = output + str(arr[i])  
    if i < len(arr) - 1:  
        output = output + " "  
    i = i + 1  
print(output)
```

Depois:

```
print(f"Array: {' '.join(map(str, arr))}")
```

- Otimização Peephole

A ideia consiste em caso o compilador encontre uma sequência de instruções que pode ser substituída por uma sequência mais curta ou mais rápida, ele faz essa troca.

Então ao colocar as expressões:

```
KB = 1024  
MB = 1024 * KB  
GB = 1024 * MB  
LARGEST_CACHE_SZ = 8 * MB  
ARRAY_SIZE = 1000  
NUM_ITERATIONS = 100
```

Serão trocados por:

```
# O bytecode gerado se parece mais com isso:  
KB = 1024  
MB = 1048576 # Já pré-calculado  
GB = 1073741824 # Já pré-calculado  
LARGEST_CACHE_SZ = 8388608 # Já pré-calculado  
ARRAY_SIZE = 1000  
NUM_ITERATIONS = 100
```

Um programa que você já desenvolveu previamente em Python

Para essa etapa, foi-se utilizado um programa em python contendo duas funções: uma versão otimizada e uma não otimizada do clássico algoritmo que calcula a sequência de fibonacci.

Usando o `timeit`, cada função foi executada 5 vezes calculando o valor de `fibonacci(40)`. Os tempos de execução obtidos com o código sem e com otimização estão apresentados abaixo:

```
=== Resultados Finais (Tempo Total) ===
Não Otimizado = 165.93161049300033
Otimizado = 0.00003420399980313960

=== Resultados Finais (Tempo Médio) ===
Não Otimizado = 33.186322098600066
Otimizado = 0.00000684079996062792
```

Otimizações adotadas:

- Utilização de **Cache**:

A principal otimização adotada foi a utilização de um algoritmo de cache para evitar recalculiar estados repetidos durante a execução da recursão. Para isso, utilizamos a biblioteca `functools` que possui o decorator `@functools.cache` que mantém referências para os argumentos e, caso a função seja chamada em outro momento com os mesmos parâmetros, a cache apenas retorna o resultado pré-calculado, evitando uma quantidade massiva de processamento desnecessário.

Versão não otimizada:

```
def fibonacci_unoptimized(n):
    if n <= 1:
        return max(n, 0)

    return fibonacci_unoptimized(n-1) + fibonacci_unoptimized(n-2)
```

Versão otimizada:

```
@functools.cache
def fibonacci_optimized(n):
    if n <= 1:
        return max(n, 0)

    return fibonacci_optimized(n-1) + fibonacci_optimized(n-2)
```